

Essentials of Data Science

[FYBTech - CS104]

By

Team — Essentials of Data Science

School of Computer Engineering,
MIT Academy of Engineering, Alandi(D.)

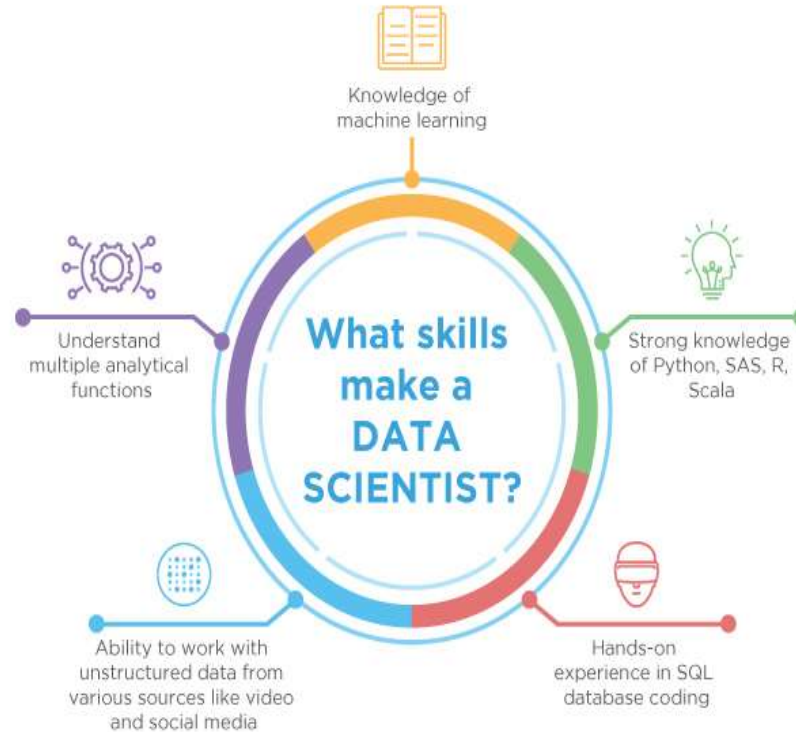
Need of Data science

- **Data science** is an interdisciplinary field that uses scientific methods, processes, algorithms and domain knowledge to extract insights from structured as well as unstructured data to solve complex problems across a broad range of application domains.

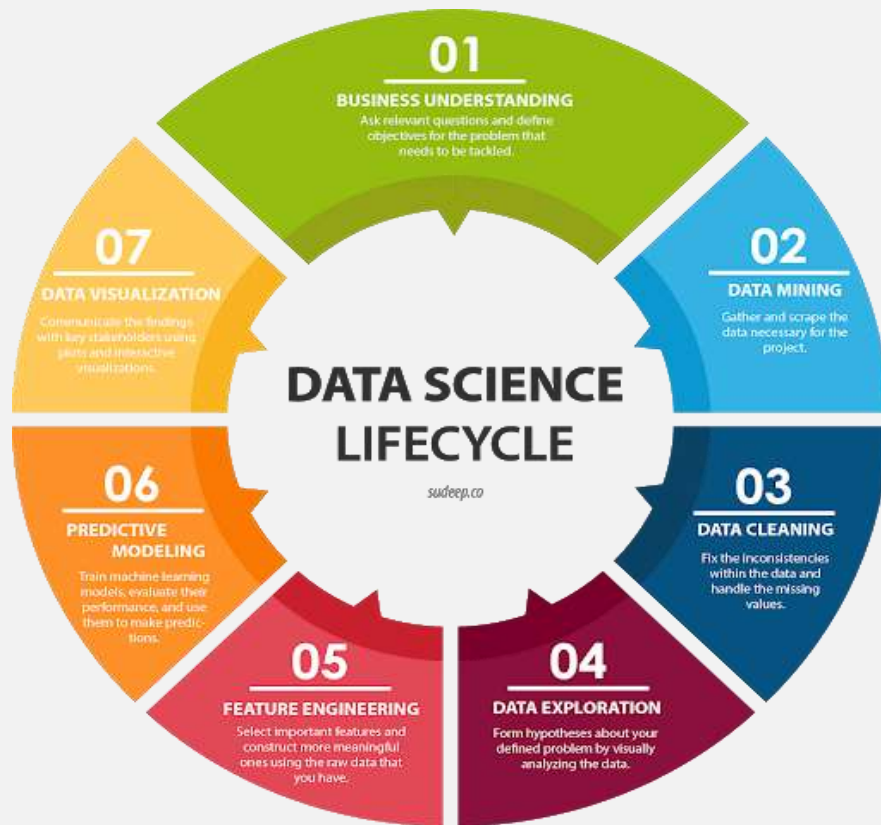
Why Data science is Important ?

- According to IDC, by 2025, global data will grow to 175 zetta-bytes.
- **Data Science enables** companies to efficiently **understand gigantic data** from multiple sources and **derive valuable insights** to make **smarter data-driven decisions**.
- Data Science is widely used in various industry domains, including marketing, healthcare, finance, banking, policy work, and more.

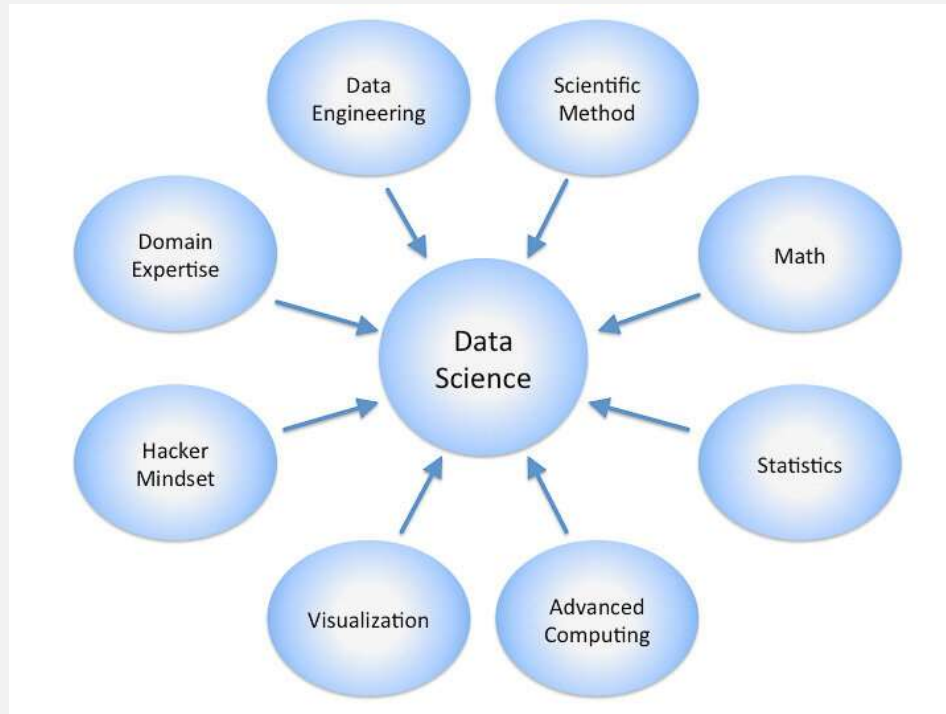
Data science Skill-Set



Data Science Life Cycle



Insight of Data Science



Applications of Data Science



Importance of Course

- **Basic Concepts of Data Science**
- **Basics for Multidisciplinary Projects**
- **Competitive Coding for every student**
- **Understanding of simple data science techniques**
- **Basic programming language**

Details of Course

Name of the Course	Essentials of Data Science
Semester in which the Course will be taught	Second Semester (F. Y. B. Tech)
Course Code	CS104
Theory Sessions	02 Hrs. / Week
Practical Sessions	04 Hrs. / Week
Course Credit	04

Details of Course

Mid-Semester Examination	20 Marks	Total Theory marks 75
End Semester Examination	40 Marks	
Internal Assessment	15 Marks	
Continuous Assessment & Mid-Semester Practical Examination	50 Marks	Total Practical marks 75
Practical Exam	25 Marks	
Total Marks	150 Marks	150

Course Objectives & Outcomes

Course Objectives

- 1: To get familiar with the basics of Python programming.
- 2: To learn different data structures in Data Science.
- 3: To use data computation methods in Data Science
- 4: To introduce data manipulation methods in Data Science.
- 5: To use data visualization methods in Data Science.
- 6: To understand the importance and basic techniques of data science.

Course Outcomes

- 1: Build a python program for handling syntax and semantics. [Applying]
- 2: Demonstrate proficiency in handling data structures useful in Data Science.[Applying]
- 3: Apply the different methods of data computations on real time data.[Applying]
- 4: Interpret the different methods of data manipulation on real time data.[Applying]
- 5: Apply data visualization for real time data.[Applying]
- 6:To apply basic techniques of data science. [Applying]

Course Contents

Unit I: Python Fundamentals for Data Science

Unit II: Data Organizational Structures

Unit III: Data Computation

Unit IV: Data Manipulation

Unit V: Data Distribution and Visualization

Unit VI: Problem Analysis in Data Science

Books

TEXT BOOK

- Jake VanderPlas, Python Data Science Handbook: Essential Tools for Working with Data Paperback, O'Reilly Media, ISBN:9781491912133, 1491912138
- Wes McKinney, Python for Data Analysis Data Wrangling with Pandas, NumPy, and IPython, O'Reilly Media, ISBN: 9781491957615, 1491957611
- Wes McKinney, Python for Data Analysis, O'Reilly Media, Inc., 2017, ISBN: 9781491957660
- Clinton W. Brownley, Foundations for Analytics with Python From Non-Programmer to Hacker, O'reilly, ISBN: 9781491922484, 1491922486

REFERENCE BOOK

- John Paul Mueller, Luca Massaron, Python for Data Science For Dummies, Wiley, ISBN: 9781119547624, 1119547628
- Joel Grus, Data Science from Scratch, O'Reilly Media, Inc., 2019, ISBN: 9781492041085
- Frank Kane, Hands-On Data Science and Python Machine Learning, Packt Publisher, ISBN: 9781787280229, 1787280225
- Samir Madhavan, Mastering Python for Data Science, Packt Publisher, ISBN: 9781784392628, 1784392626

Internal Assessment

Activity No. 1 – Unit 1 & 2 – Video Making (10 Marks)

Assignment No. 1 – Unit 3 & 4 – Forming Statements (10 Marks)

Activity No. 2 – Unit 5 & 6 – Quiz (10 Marks)

Continuous Assessment

Codechef Assignments – 20 Marks

Cisco Certification for Essentials of Python – 20 Marks

Laboratory Assignments – 20 Marks

Project Demonstration – 30 Marks

Mid Semester Practical Exam – 25 Marks

Github Resource Creation – 10 Marks

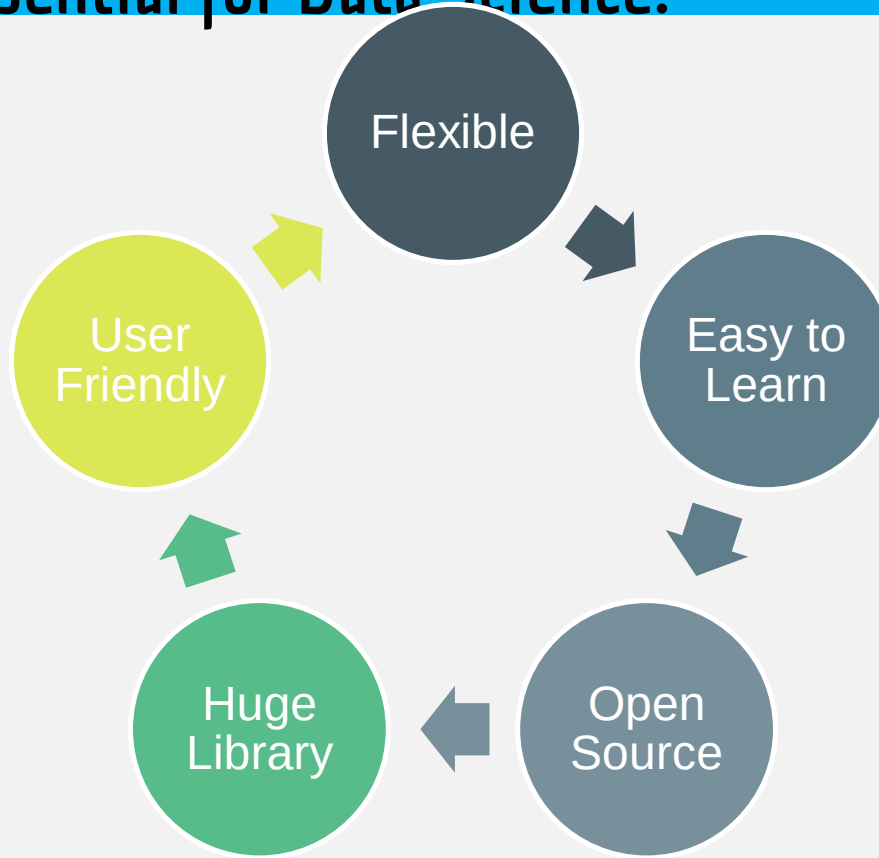
Throry and Practical – Course Instructors

Sr. No.	Faculty Name	Experience	Mail ID	Division/ Batch
1	Shubhangi Kale	19	spkale@mitaoe.ac.in	B Division
2	Prajakta Ugale	19	pvugale@mitaoe.ac.in	B1, B3
3	Shubhangi Kale	19	spkale@mitaoe.ac.in	B2, B4

Learn
SOMETHING
NEW
every day

designed by  **vexels**

Why Python is essential for Data Science?



Introduction to Python Fundamentals

Simply Easy to Learn

Scalable

Interpreted Language

OOP Support

Strong and Large Typing

Cross-Platform Compatibility

Extensive Third-Party Libraries:

Community Support

Data types

- Mutable Data types – if its value can be changed after it is created
 - Lists- `lst = [ 'Pune', 'Nashik', 'Mumbai' ]`
 - Dictionaries - `dict1 = { 'Pune': 38, 'Nashik': 36, 'Mumbai': 35.6 }`
 - Sets - `city = { 'Pune', 'Nashik', 'Mumbai' }`
- Immutable data type – if its value can not be changed once it is created
 - Integers - `Temperature = 38`
 - Floats - `Temperature = 35.6`
 - Booleans - `Hot_City = True`
 - Strings - `City_Name = 'Pune'`
 - Tuples - `city_name = ("apple", "banana", "cherry")`

Data Conditioning

Relational Operator

- Equals: `city_name1 == city_name2`
- Not Equals: `city_name1 != city_name2`
- Less than: `temperature1 < temperature2`
- Less than or equal to: `temperature1 <= temperature2`
- Greater than: `temperature1 >= temperature2`
- Greater than or equal to: `temperature1 >= temperature2`

Logical Operator

- **and** – `( temperature1 < temperature2 and temperature1 < temperature3 )`
- **or** – `( temperature1 < temperature2 or temperature1 < temperature3 )`
- **not** - `hot_city= True`
`not(hot_city)`

Data Conditioning

Bitwise Operator

- `temperature1 = 100010 (34)`
- `temperature2 = 100011 (36)`
- **bitwise and (&) operator** - `temperature1 & temperature2`
- **bitwise or (|) operator** - `temperature1 | temperature2`
- **bitwise not (~) operator** - `~ temperature1`
- **bitwise xor (^) operator** - `temperature1 ^ temperature2`

Bitwise shifting operator

- `temperature1 = 0010 0010 (34)`
- **bitwise left shift (<<) operator** - `temperature1 << 1`
- **bitwise right shift (>>) operator** - `temperature1 >> 1`

Importance of Data Conditioning

Data conditioning plays a vital role in the field of data science, as it is a necessary step in the process of preparing data for analysis such as

- Ensures data quality
- Increases efficiency
- Enhances data modeling
- Facilitates data analysis
- Supports reproducibility

Data Conditioning using Decision Statements

- **if statement**

Syntax:

if condition:

 # code to be executed if the condition is True

- **if-else statement**

Syntax:

if condition:

 # code to be executed if the condition is True

else:

 # code to be executed if the condition is False

Data Conditioning using Decision Statements

- **if-elif-else statement**

Syntax:

if condition1:

 # code to be executed if condition1 is True

elif condition2:

 # code to be executed if condition2 is True

elif condition3:

 # code to be executed if condition3 is True

else:

 # code to be executed if all conditions are False

Data Conditioning using Decision Statements

Example 1:

```
weather = input("What is the weather today? ") #input
if weather == "Sunny":
    print("It is a great day to go for a picnic!")
```

Example 2:

```
weather = input("What is the weather today? ") #input
if weather == "Sunny":
    print("It is a great day to go for a picnic!")
else
    print("Sorry, I don't understand that weather condition.")
```

Data Conditioning using Decision Statements

Example 3:

```
weather = input("What is the weather today? ") #input
if weather == "Sunny":
    print("It is a great day to go for a picnic!")
elif weather == "Cloudy":
    print("Don't forget to bring a jacket in case it rains.")
elif weather == "Rainy":
    print("Remember to bring an umbrella!")
else:
    print("Sorry, I don't understand that weather condition.")
```

Importance of Data Conditioning using Decision Statements

- Decision statements are a fundamental component of data science
- It is useful for data scientists to control the flow of data in their program
- Data Scientist make data-driven decisions based on specific conditions
- It uses to filter data for data analysis
- It uses in data preprocessing such missing values, noisy values and inconsistent values
- It uses in data visualization to select specific condition

Data Conditioning using Iterative Statements

- **While loop**

Syntax:

while condition:

 # code block to be executed while the condition is True

- **For loop**

Syntax:

for variable in iterable:

 # code block to be executed for each item in the iterable

Data Conditioning using Iterative Statements

- **While loop – Find Max Temperature**

Syntax:

while condition:

code block to be executed while the condition is True

- **For loop**

Syntax:

for variable in iterable:

code block to be executed for each item in the iterable

Importance of Data Conditioning using Iterative Statements

- Iterative statements, such as loops, are essential in data science as they allow us to repeat a set of instructions multiple times. The specific use of data conditioning using iterative statements as
 - Data preprocessing
 - Feature engineering
 - Model Training
 - Model Evaluation

Functions: Basics of functions

- A function is a block of code which only runs when it is called.
- We can pass data, known as parameters, into a function.
- A function can return data as a result.

- Syntax:
- `def function(): #definition of function`
`print("Hello World")`

`function()` #call of function

Functions: Basics of functions

```
def max_temperature(temp1, temp2):  
    if(temp1 > temp2):  
        max = temp1  
    else:  
        max = temp2  
    return max
```

```
result = max_temperature (30, 35)  
print(result)
```

Importance of Functions

- Functions are used to organize code
- It makes program more modular, reusable
- It is easier to read and maintain program
- It can be used to automate repetitive tasks, reduce the chances of errors, and make the code more efficient.
- Functions in data science can be used for a wide range of tasks, such as data preprocessing, data cleaning, data transformation, feature engineering, model training, and model evaluation.
- Wide range of functions are using in Data Processing such as data preprocessing, data cleaning, data transformation etc.
- Function helps data scientist to write a code in systematic way

Variable scope and lifetime

- Local, Global and Non-Local Variable help control the scope and visibility of variables in a program.
- It is essential for writing efficient and organized code in data science
- Local Variable- hen we define a function, we can create variables which are scoped only to that function and are not accessible or visible outside of the function to the compiler. These variables are referred to as ***local variables***

```
def my_function():  
    local_variable = 50  
    print (local_variable)  
my_function()
```

- if we attempt to access local_variable outside the function, then it will generate an error.

Variable scope and lifetime

- **Global Variable:** Global variables are those which are defined outside a function and can be accessed throughout the program. They have a global scope and can be accessed from any part of the program, including functions.

Example:

```
1)      temperature = 50
        def my_function():
            print (temperature)
        print (temperature)
```

```
2)      temp = 50
        def my_function():
            global temp
            temp = temp + 1
            print (temp)
```

```
my_function()
print (temp)
```

Variable scope and lifetime

- Non-Local Variable - Those which are defined in a nested function and are used in the enclosing function. They are neither local nor global variables.
- They are declared using the **nonlocal** keyword.
- Example:

```
def outer_function():  
    x = "local"  
  
    def inner_function():  
        nonlocal x  
        x = "nonlocal"  
        print("Value of x inside the inner function:", x)  
    inner_function()  
    print("Value of x inside the outer function:", x)  
outer_function()
```

Lambda or anonymous function Functions

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.
- Functions in Python that can be defined without a name.
- They are commonly used in for flexibility, readability, performance, functional programming and simplicity,

- Syntax: *lambda arguments : expression*

- *Example:*

1) `Inc_temp = lambda temp : temp + 10`

`print(Inc_temp(5))`

2) `t = lambda t1, t2, t3 : t1 + t2 + t3`

`print(x(5, 6, 2))`

Modules: Introduction to modules

- Modules are the pre-defined files that contain the python codes which depict the basic functionalities of class, methods, variables, etc.
- It consists of different functions, classes in a group of files inside a directory. Modules can also be termed as Libraries.
- These are basically the pre-defined methods that can be used to make the code more efficient and reduce redundancy.
- Modules bind the code and reduce the repetitions of functions frequently used in the code.
- Thus, it makes the code much clear and easy to understand.
- Mechanism for Python Module:



- The moment a module is imported through a program, Python Interpreter fetches the module from either of the following locations:
 - Program Directory
 - The directory in the PYTHONPATH variable
 - Default directory

Modules: Introduction to modules

- The list of available modules in Python can be found by `help("module")`

Importing modules from Python Standard path

Syntax:

```
import module_name
```

Example:

```
import math  
print (math.sqrt(121))  
print (math.pi)  
print (dir(math))
```


Python Packages and Standard Library Modules

Difference between a module and a package in Python

Python Module: These are set of pre-defined files that contain the python codes which depict the basic functionalities of class, methods, variables, etc.

Python Package: It is a directory that holds and contains modules and sub-packages.

Data Handling using Files.

Open Operation

```
f = open(filename, mode)
```

Where the following mode is supported:

- 1.r:** open an existing file for a read operation.
- 2.w:** open an existing file for a write operation. If the file already contains some data then it will be overridden but if the file is not present then it creates the file as well.
- 3.a:** open an existing file for append operation. It won't override existing data.
- 4. r+:** To read and write data into the file. The previous data in the file will be overridden.
- 5.w+:** To write and read data. It will override existing data.
- 6.a+:** To append and read data from the file. It won't override existing data.

Data Handling using Files.

read() : This function reads the entire file and returns a string

readline() : This function reads lines from that file and returns as a string. It fetch the line n, if it is been called nth time.

readlines() : This function returns a list where each element is single line of that file.

readlines() : This function returns a list where each element is single line of that file.

write() : This function writes a fixed sequence of characters to a file.

writelines() : This function writes a list of string.

append() : This function append string to the file instead of overwriting the file.

Data Handling using Files.

```
# a file named "geek", will be opened with  
the reading mode.  
file = open('abc.txt', 'r')  
# This will print every line one by one in  
the file  
for each in file:  
    print (each)
```

```
# Python code to create a file  
file = open('abc.txt','w')  
file.write("This is the write command")  
file.write("It allows us to write in a  
particular file")  
file.close()
```

```
file = open('abc.txt', 'a')  
file.write("This will add this line")  
file.close()
```

Data Handling using Files.

`rstrip()`: This function strips each line of a file off spaces from the right-hand side.

`lstrip()`: This function strips each line of a file off spaces from the left-hand side.

```
# Python code to illustrate with()  
with open("file.txt") as file:  
    data = file.read()  
# do something with data
```

```
# Python code to illustrate with()  
alongwith write()  
with open("file.txt", "w") as f:  
    f.write("Hello World!!!")
```

```
# Python code to illustrate split()  
function  
with open("file.text", "r") as file:  
    data = file.readlines()  
    for line in data:  
        word = line.split()  
        print (word)
```

Unit II

Data Organizational Structures

By

Team — Essentials of Data Science

School of Computer Engineering,
MIT Academy of Engineering, Alandi(D.)

Categories of Data

- **Structured data** — typically categorized as quantitative data — is highly organized and easily decipherable by [machine learning algorithms](#). Structured query language (SQL) is the programming language used to manage structured data
- **Pros**
 - Easily used by machine learning (ML) algorithms:
 - Easily used by business users
 - Accessible by more tools
- **Cons**
 - Limited usage:
 - Limited storage options
- **Tools:** SQLite, PostgreSQL, OLAP, MySQL

Categories of Data

Unstructured data, typically categorized as qualitative data, cannot be processed and analyzed via conventional data tools and methods. Since unstructured data does not have a predefined data model, it is best managed in [non-relational \(NoSQL\) databases](#).

Pros

- Native format
- Fast accumulation rates
- Data lake storage

Cons

- Requires expertise
- Specialized tools
- Tools:
- MongoDB, Hadoop, Azure

Categories of Data

Semi-structured data (e.g., JSON, CSV, XML) is the “bridge” between structured and unstructured data. It does not have a predefined data model and is more complex than structured data, yet easier to store than unstructured data.

Semi-structured data uses “metadata” (e.g., tags and semantic markers) to identify specific data characteristics and scale data into records and preset fields. Metadata ultimately enables semi-structured data to be better cataloged, searched and analyzed than unstructured data.

Categories of Data

Research has two main data types:

1. Categorical data-- Categorical data refers to a form of information that can be stored and identified based on their names or labels. It is a type of qualitative data that can be grouped into categories instead of being measured numerically.

- Example: Birthdate
- Favorite sport
- Hair Color
- Height

There are two kinds of categorical data:

1. Nominal Data
2. Ordinal Data

Categories of Data

There are two kinds of categorical data:

1. Nominal Data-- Nominal data is a type of data that consists of categories that can't be ordered or ranked

Example: symbols, words, letters, and the gender of a person.

2. Ordinal Data-- Ordinal data is a category of data that has a natural order. It is often used in surveys, questionnaires, and the fields of finance and economics. Ordinal data stands out since it is impossible to differentiate between data values.

Clothing sizes are one example of this type of data (small, medium, and large are not measurable differences, but they are clearly ordered to show size comparisons).

Features of Categorical Data

- Categories -- There are two types of categorical data: nominal data and ordinal data.
- Qualitativeness -- It is qualitative. In other words, it uses a string of words instead of numbers to describe an event.
- Nature -- Depending on its nature, categorical data can also be divided into binary and non-binary categories. A binary question has two possible answers, such as yes or no, while a non-binary question would have more than two answers, such as maybe.
- Numeric values -- It is qualitative, yet it often includes numerical values. But these values.
- Graphical analysis -- A pie chart and bar chart can both be used to analyze it visually.
- Analysis
The mode and median tools are used to analyze categorical data.

Categories of data

2. Numerical Data -- Numerical data is a data type expressed in numbers, rather than natural language description. Sometimes called quantitative data, numerical data is always collected in number form.

There are

- **Discrete Data**

Discrete Data represents countable items and can take both numerical and categorical forms, depending on usage. It takes on values that can be grouped into a list, where the list may either be finite or infinite. Whether finite or infinite, discrete data take on counting numbers like 1 to 10 or 1 to infinity,

- **Continuous Data:**

This is a type of numerical data which represents measurements—their values are described as intervals on a real number line, rather than take counting numbers. Ex. Score is 4.495, 2.125

There are two types of Continuous data i.e. interval and ratio data

Categories of data

- **Interval Data**

This is a data type measured along a scale, in which each point is placed at an equal distance from one another.

- **Ratio Data**

Ratio data is a continuous data type similar to interval data but has a zero point

For ratio data, the temperature may not only be measured in degrees Celsius and degrees Fahrenheit, but also in Kelvin

Features of Numerical Data

- **Categories:** There are two main categories of numerical data, namely; discrete and continuous data.
- **Quantitativeness:** Numerical data is sometimes called quantitative data due to its quantitative nature.
- **Arithmetic Operation:** One can perform arithmetic operations like addition and subtraction on numerical data.
- **Estimation & Enumeration:** Numerical data can both be estimated and enumerated. for instance, a 4.495623 CGPA is rounded up to 4.50.
- **Interval Difference:** The difference between each interval on a numerical data scale are equal. For example, the difference between 5 minutes and 10 minutes on a wall clock is the same as the difference between 10 and 15 minutes.
- **Analysis:** Numerical data is analyzed using descriptive and inferential statistical methods, depending on the aim of the research.
- **Data Visualisation:**

Time Series Data

•Time series data is a collection of observations obtained through **repeated measurements over time**. Plot the points on a graph, and one of your axes would always be time

Examples of time series analysis:

- Electrical activity in the brain
- Rainfall measurements
- Stock prices
- Number of sunspots
- Annual retail sales
- Monthly subscribers
- Heartbeats per minute

List

- **Python Lists** are just like dynamically sized arrays, declared in other languages (vector in C++ and ArrayList in Java). In simple language, a list is a collection of things, enclosed in [] and separated by commas.

```
Var = ["Data", "Essentials", "Science"]  
print(Var)
```

- Lists in Python can be created by just placing the sequence inside the square brackets []. Unlike Sets, a list doesn't need a built-in function for its creation of a list.

List

```
# Creating a List
List = []
print("Blank List: ")
print(List)

# Creating a List of numbers
List = [10, 20, 14]
print("\nList of numbers: ")
print(List)

# Creating a List of strings and accessing
# using index
List = ["Data", "Essentials", "Science"]
print("\nList Items: ")
print(List[0])
print(List[2])
```

Creating List with multiple elements

```
# Creating a List with
# the use of Numbers
# (Having duplicate values)
List = [1, 2, 4, 4, 3, 3, 3, 6, 5]
print("\nList with the use of Numbers: ")
print(List)

# Creating a List with
# mixed type of values
# (Having numbers and strings)
List = [1, 2, 'Data', 4, 'For', 6, 'Science']
print("\nList with the use of Mixed Values: ")
print(List)
```

Assessing elements from the list

```
List = ["data", "science", "essentials"]  
# accessing a element from the  
# list using index number  
print("Accessing a element from the list")  
print(List[0])  
print(List[2])
```

Assessing elements from the multi dimensional list

```
# Creating a Multi-Dimensional List
# (By Nesting a list inside a List)
List = [['Date', 'For'], ['Science']]

# accessing an element from the Multi-Dimensional List using
# index number
print("Accessing a element from a Multi-Dimensional list")
print(List[0][1])
print(List[1][0])
```

Negative Indexing

```
List = [1, 2, 'Data', 4, 'For', 6, 'Science']  
  
# accessing an element using  
# negative indexing  
print("Accessing element using negative indexing")  
  
# print the last element of list  
print(List[-1])  
  
# print the third last element of list  
print(List[-3])
```

Input Accepting

```
List = [1, 2, 'Data', 4, 'For', 6, 'Science']  
Print(len(List))  
string = input("Enter elements (Space-Separated): ")
```

```
# split the strings and store it to a list  
lst = string.split()  
print('The list is:', lst)    # printing the list
```

```
rows, cols = (5, 5)  
arr = [[0]*cols]*rows  
print(arr)
```

Input Accepting

```
thislist = list(("data", "science", "essentials")) # note the  
double round-brackets  
print(thislist)
```

```
thislist =  
["apple", "banana", "cherry", "orange", "kiwi", "mango"]  
thislist[1:3] = ["blackcurrant", "watermelon"]  
print(thislist)
```

```
thislist = ["apple", "banana", "cherry"]  
thislist.append("orange")  
print(thislist)
```


List Operations

```
thislist = ["apple", "banana", "cherry"]  
thislist.insert(1, "orange")  
print(thislist)
```

```
thislist = ["apple", "banana", "cherry"]  
tropical = ["mango", "pineapple", "papaya"]  
thislist.extend(tropical)  
print(thislist)
```

```
thislist =  
["apple", "banana", "cherry"]  
thislist.remove("banana")  
print(thislist)
```

```
thislist =  
["apple", "banana", "cherry"]  
thislist.pop(1)  
print(thislist)
```

List Operations

```
thislist = ["apple", "banana", "cherry"]  
del thislist[0]  
print(thislist)
```

```
thislist = ["apple", "banana", "cherry"]  
thislist.clear()  
print(thislist)
```

List Comprehension: List Comprehension offers the shortest syntax for looping through lists:

```
thislist = ["apple", "banana", "cherry"]  
[print(x) for x in thislist]
```

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]  
thislist.sort(reverse = True)  
print(thislist)
```

List Operations

```
def myfunc(n):  
    return abs(n - 50)
```

```
thislist = [100, 50, 65, 82, 23]  
thislist.sort(key = myfunc)  
print(thislist)
```

```
thislist =  
["banana", "Orange", "Kiwi", "cherry"]  
thislist.sort(key = str.lower)  
print(thislist)
```

```
thislist =  
["banana", "Orange", "Kiwi", "cherry"]  
thislist.reverse()  
print(thislist)
```

List Operations

```
thislist = ["apple", "banana", "cherry"]  
mylist = thislist.copy()  
print(mylist)
```

```
list1 = ["a", "b", "c"]  
list2 = [1, 2, 3]
```

```
list3 = list1 + list2  
print(list3)
```

```
list1 = ["a", "b", "c"]  
list2 = [1, 2, 3]
```

```
for x in list2:  
    list1.append(x)
```

```
print(list1)
```

```
list1 = ["a", "b", "c"]  
list2 = [1, 2, 3]
```

```
list1.extend(list2)  
print(list1)
```

List Operations

<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

List Operations

<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

Stack

A stack is a linear data structure in python in which addition and deletion of elements can be done at one end only.

A stack is known as LIFO (Last-In, First-out) data structure in python.

LIFO means the elements which are added in the last would be first one to remove.

Example of stack are pile of books, pile of plates or stack of carom coins.

Stack

There are two main operations on stack

Push-Addition of elements on the Top of the Stack is called PUSH

Pop- deletion of elements from the Top of stack is called as POP

Queue

There are two main operations on stack

Push-Addition of elements on the Top of the Stack is called PUSH

Pop- deletion of elements from the Top of stack is called as POP

Queue

Queue is a specialized data structure in which elements are added at one end called Rear and Removed from other end called Front.

Operations on Queue

- 1) **Insert**: In this a new element is added at the end of the list called Rear.
- 2) **Delete**: In this an element is to be deleted from one end called Front.

example: outside ATM, In BANK, Buying ticket in Movie Hall

Queue

Queue is a specialized data structure in which elements are added at one end called Rear and Removed from other end called Front.

Operations on Queue

- 1) **Insert**: In this a new element is added at the end of the list called Rear.
- 2) **Delete**: In this an element is to be deleted from one end called Front.

example: outside ATM, In BANK, Buying ticket in Movie Hall

Tuple

A Tuple is a collection of Python objects separated by commas. In some ways, a tuple is similar to a list in terms of indexing, nested objects, and repetition but a tuple is immutable, unlike lists that are mutable.

The empty tuple can be created as

```
T=()
```

Example

```
T1=(10,20,30,40)
```

```
T2=('A','B','C','D')
```

```
T3=('A',10,20)
```

```
T4=('aaa','bbb','ccc','ddd')
```

Tuple

```
thistuple = ("apple", "banana", "cherry")  
print(len(thistuple))
```

```
thistuple = ("apple",)  
print(type(thistuple))
```

```
#NOT a tuple  
thistuple = ("apple")  
print(type(thistuple))
```

```
tuple1 = ("apple", "banana", "cherry")  
tuple2 = (1, 5, 7, 9, 3)  
tuple3 = (True, False, False)
```

Tuple

```
tuple1 = ("abc", 34, True, 40, "male")
```

```
thistuple = tuple(("apple", "banana", "cherry")) # note  
the double round-brackets  
print(thistuple)
```

```
thistuple = ("apple", "banana", "cherry")  
print(thistuple[1])
```

```
thistuple = ("apple", "banana", "cherry")  
print(thistuple[-1])
```

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")  
print(thistuple[2:5])
```

Tuple

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")  
print(thistuple[:4])
```

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")  
print(thistuple[2:])
```

Convert the tuple into a list to be able to change it:

```
x = ("apple", "banana", "cherry")  
y = list(x)  
y[1] = "kiwi"  
x = tuple(y)  
  
print(x)
```

Tuple

Convert the tuple into a list, add "orange", and convert it back into a tuple:

```
thistuple = ("apple", "banana", "cherry")  
y = list(thistuple)  
y.append("orange")  
thistuple = tuple(y)
```

#Create a new tuple with the value "orange", and add that tuple:

```
thistuple = ("apple", "banana", "cherry")  
y = ("orange",)  
thistuple += y
```

```
print(thistuple)
```

#Unpack Tuples

```
fruits = ("apple", "banana", "cherry")
```

```
(green, yellow, red) = fruits
```

```
print(green)  
print(yellow)  
print(red)
```


Tuple

```
thistuple = ("apple", "banana", "cherry")  
for x in thistuple:  
    print(x)
```

```
thistuple = ("apple", "banana", "cherry")  
for i in range(len(thistuple)):  
    print(thistuple[i])
```

```
tuple1 = ("a", "b" , "c")  
tuple2 = (1, 2, 3)
```

```
tuple3 = tuple1 + tuple2  
print(tuple3)
```

[count\(\)](#)

Returns the number of times a specified value occurs in a tuple

[index\(\)](#)

Searches the tuple for a specified value and returns the position of where it was found

Dictionary

Dictionary: in Python is an ordered collection of data values, used to store data values like a map, which, unlike other Data Types that hold only a single value as an element.

Dictionary holds **key:value** pair. Key-value is provided in the dictionary to make it more optimized.

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

#dictionary Item

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
print(thisdict["brand"])  
  
print(len(thisdict))
```

Dictionary

```
thisdict = dict(name = "John", age = 36, country = "Norway")  
print(thisdict)
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = thisdict["model"]
```

```
x = thisdict.get("model")
```

```
x = thisdict.keys()
```

```
x = thisdict.values()
```

Dictionary

Add a new item to the original dictionary, and see that the keys list gets updated as well:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.keys()
```

```
print(x) #before the change
```

```
car["color"] = "white"
```

```
print(x) #after the change
```

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

```
x = car.values()
```

```
print(x) #before the change
```

```
car["color"] = "red"
```

```
print(x) #after the change
```

Dictionary

```
x = thisdict.items()
```

```
#update
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.update({"year": 2020})
```

```
#popitem
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.popitem()  
print(thisdict)
```

```
#pop
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.pop("model")  
print(thisdict)
```

```
#delete
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
del thisdict["model"]  
print(thisdict)
```

Dictionary

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
del thisdict
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.clear()  
print(thisdict)
```

```
#Loop  
for x in thisdict:  
    print(x)
```

```
#Loop  
for x in thisdict:  
    print(thisdict[x])
```

```
#Loop  
for x in thisdict.values():  
    print(x)
```

```
#Loop  
for x in thisdict.keys():  
    print(x)
```

```
#Loop  
for x, y in thisdict.items():  
    print(x, y)
```

Dictionary

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
mydict = thisdict.copy()  
print(mydict)
```

```
myfamily = {  
    "child1" : {  
        "name" : "Emil",  
        "year" : 2004  
    },  
    "child2" : {  
        "name" : "Tobias",  
        "year" : 2007  
    },  
    "child3" : {  
        "name" : "Linus",  
        "year" : 2011  
    }  
}  
  
print(myfamily["child2"]["name"])
```

Dictionary -- setdefault

Add an item if the key does not exist in dict with setdefault in Python

The `dict.setdefault()` method returns the value of the specified key in the dictionary. If the key is not found, then it adds the key with the specified default value.

If the default value parameter is not specified, then it sets `None`.

Dictionary -- setdefault

`dict.setdefault(key, defaultValue)`

Parameters:

1. key: Required. The key to be searched for.
2. defaultValue: Optional. If the key is not found, it is inserted in the dictionary with this defaultValue. By default, it is None.

Return Value:

Returns the value of the key if the key is present.

Returns the defaultValue if the key is not present or None if defaultValue is not specified.

The following example demonstrates the dict.setdefault().

Example1: setdefault()

```
romanNums = {'I':1, 'II':2, 'III':3, 'IV':4, 'V':5 }
```

```
value = romanNums.setdefault('I')
```

```
print("The return value is: ", value)
```

Output

The return value is: 1

Dictionary -- setdefault

If the key is not found, it is inserted in the dictionary with this default value. By default, it is None.

Example2: setdefault() Copy

```
romanNums = {'I':1, 'II':2, 'III':3}  
value = romanNums.setdefault('IV')
```

```
print("The return value is: ",value)  
print("Updated dictionary: ",romanNums)
```

Output

```
The return value is: None Updated dictionary:  
{'I': 1, 'II': 2, 'III': 3, 'IV': None}
```

In the above example, the specified key is not present in the dictionary, so when the method is called, the key is added to the dictionary, with a value of None

Dictionary -- setdefault

If the key is not found, it is inserted in the dictionary with this default value. By default, it is None.

Example2: setdefault() Copy

```
romanNums = {'I':1, 'II':2, 'III':3}  
value = romanNums.setdefault('IV')
```

```
print("The return value is: ",value)  
print("Updated dictionary: ",romanNums)
```

Output

```
The return value is: None Updated dictionary:  
{'I': 1, 'II': 2, 'III': 3, 'IV': None}
```

In the above example, the specified key is not present in the dictionary, so when the method is called, the key is added to the dictionary, with a value of None

Dictionary -- setdefault

If the key passed as the argument is not present in the dictionary, the key is inserted into the dictionary with the specified value.

Example: Add Key-Value Copy

```
romanNums = {'I':1, 'II':2, 'III':3 }
```

```
value = romanNums.setdefault('IV', 4)
```

```
print("The return value is: ",value)
```

```
print("Updated dictionary: ",romanNums)
```

Output

The return value is: 4

Updated dictionary: { 'I': 1, 'II': 2, 'III': 3, 'IV': 4 }

Dictionary -- json

JSON stands for JavaScript Object Notation. It means that a script (executable) file which is made of text in a programming language, is used to store and transfer the data. Python supports JSON through a built-in package called json. To use this feature, we import the json package in Python script. The text in JSON is done through quoted-string which contains value in key-value mapping within { }. It is similar to the dictionary in Python.

Function Used:

- **json.load():** json.loads() function is present in python built-in 'json' module. This function is used to parse the JSON string.

Dictionary -- json

Data.json

```
{  
  "people1": [  
    {  
      "name": "Nikhil",  
      "website": "gfg.com",  
      "from": "Delhi"  
    },  
    {  
      "name": "Abhinav",  
      "website": "google.com",  
      "from": "Mumbai"  
    }  
  ],  
  "people2": [  
    {  
      "name": "Anshul",  
      "website": "apple.com",  
      "from": "Chennai"  
    }  
  ]  
}
```

Dictionary -- json

```
# importing the module
import json

# Opening JSON file
with open('data.json') as json_file:
    data = json.load(json_file)

# Print the type of data variable
print("Type:", type(data))

# Print the data of dictionary
print("\nPeople1:", data['people1'])
print("\nPeople2:", data['people2'])
```

Set

A Set is an unordered collection data type that is iterable, mutable, and has no duplicate elements. Python's set class represents the mathematical notion of a set.

Sets are used to store multiple items in a single variable.

Set is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Tuple, and Dictionary, all with different qualities and usage.

A set is a collection which is *unordered*, *unchangeable*, and *unindexed*.

Unordered

Unordered means that the items in a set do not have a defined order.

Set items can appear in a different order every time you use them, and cannot be referred to by index or key.

Unchangeable

Set items are unchangeable, meaning that we cannot change the items after the set has been created.

Once a set is created, you cannot change its items, but you can remove items and add new items.

Duplicates Not Allowed

Sets cannot have two items with the same value.

Set

Example

#create an empty set

```
empty_set = set()
```

#create a set of integer type

```
student_id = {112, 114, 116, 118, 115}  
print('Student ID:', student_id)
```

create a set of string type

```
vowel_letters = {'a', 'e', 'i', 'o', 'u'}  
print('Vowel Letters:', vowel_letters)
```

create a set of mixed data types

```
mixed_set = {'Hello', 101, -2, 'Bye'}  
print('Set of mixed data types:', mixed_set)
```

Set

Add Items to a Set in Python

```
numbers = {21, 34, 54, 12}  
print('Initial Set:', numbers)
```

using add() method

```
numbers.add(32)  
print('Updated Set:', numbers)
```

Remove an Element from a Set

```
languages = {'Swift', 'Java', 'Python'}  
print('Initial Set:', languages)
```

remove 'Java' from a set

```
removedValue = languages.discard('Java')  
print('Set after remove():', languages)
```

Set

first set

A = {1, 3, 5}

second set

B = {1, 3, 4}

perform union operation using |

print('Union using |:', A | B)

perform union operation using union()

print('Union using union():', A.union(B))

print('Intersection using intersection():', A.intersection(B))

print('Difference using difference():', A.difference(B))

print('using symmetric_difference():', A.symmetric_difference(B))